```
pip install tensorflow
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/di
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.2
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/
Requirement already satisfied: tensorboard~=2.19.0 in /usr/local/lib/python3.12/
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: numpy<2.2.0,>=1.26.0 in /usr/local/lib/python3.12
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/d
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/loc
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/di
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.1
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-pack
```

```
import os, pathlib, shutil, random
from tensorflow import keras
from tensorflow.keras import layers
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
```

# Download and extract IMDB dataset

```
!curl -o aclImdb_v1.tar.gz https://ai.stanford.edu/~amaas/data/sentiment/aclImc
!tar -xf aclImdb_v1.tar.gz
!rm -r aclImdb/train/unsup
```

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 80.2M  100 80.2M    0     0  64.5M      0  0:00:01  0:00:01 --:--:-- 64.5M
```

```
import os
def summarize_imdb_files (base_path="aclImdb", num_files=5):
    for split in ["train", "test"]:
        print (f"\nSummary of '{split}' split:")
        for sentiment in ["pos", "neg"]:
            print (f" Sentiment: {sentiment}")
            folder_path = os.path.join(base_path, split, sentiment)
            # Ensure the directory exists before trying to list its contents
            if not os.path.exists(folder_path):
                print(f"  Directory not found: {folder_path}")
                continue
            try:
                files_list = os.listdir(folder_path)
            except FileNotFoundError:
                print(f"  Directory not found: {folder_path}")
                continue

            # Filter out non-file entries if necessary, and take top num_files
            # For this context, we assume all entries are files or we'll handle
            files_to_process = [f for f in files_list if os.path.isfile(os.path

            if not files_to_process:
                print(f"  No files found in {folder_path}")
                continue

            for i, file_name in enumerate(files_to_process):
                file_path = os.path.join(folder_path, file_name)
                try:
                    with open(file_path, "r", encoding="utf-8") as f:
                        content = f.readlines()
                    print(f"\n File {i + 1}: {file_name}")
                    print(f" Lines in file: {len(content)}")
                    print(" First 5 lines (or fewer):")
                    print("\n".join(content[:5]).strip())
                except Exception as e:
                    print(f"  Error reading file {file_name}: {e}")
```

```
summarize_imdb_files()
```

```
File 5: class_1.txt
Lines in file: 1
First 5 lines (or fewer):
```

```
 File 5: 11552_4.txt
 Lines in file: 1
 First 5 lines (or fewer):
I went into this movie expecting a thoughtfull piece about how to be accepted
```

```
# Prepare directories
batch_size = 32
base_dir = pathlib.Path("aclImdb")
val_dir = base_dir / "val"
train_dir = base_dir / "train"

for category in ("neg", "pos"):
    os.makedirs(val_dir / category, exist_ok=True)  # Allow existing directorie
    files = os.listdir(train_dir / category)
    random.Random(1337).shuffle(files)
    num_val_samples = int(0.2 * len(files))
    val_files = files[-num_val_samples:]

    for fname in val_files:
        # Only move files if they don't already exist at the destination
        if not os.path.exists(val_dir / category / fname):
            shutil.move(train_dir / category / fname, val_dir / category / fnam
```

```
# Load datasets
train_ds = keras.utils.text_dataset_from_directory(
"aclImdb/train", batch_size=batch_size
)
val_ds = keras.utils.text_dataset_from_directory(
"aclImdb/val", batch_size=batch_size
)
test_ds = keras.utils.text_dataset_from_directory (
"aclImdb/test", batch_size=batch_size
)
text_only_train_ds = train_ds.map(lambda x, y: x)
```

```
Found 20000 files belonging to 2 classes.
Found 5000 files belonging to 2 classes.
Found 25000 files belonging to 2 classes.
```

```
# Set parameters
max_length = 150
max_tokens = 10000
text_vectorization = layers.TextVectorization(
    max_tokens=max_tokens,
    output_mode="int",
    output_sequence_length=max_length
)
text_vectorization.adapt(text_only_train_ds)
```

```
# Tokenized datasets
int_train_ds = train_ds.map(
lambda x, y: (text_vectorization (x), y),
num_parallel_calls=4).take(100) # Restrict training samples to 100

int_val_ds = val_ds.map(lambda x, y: (text_vectorization(x), y),
num_parallel_calls=4).take(10000) # Restrict validation samples to 100

int_test_ds = test_ds.map(
     lambda x, y: (text_vectorization(x), y), num_parallel_calls=4)
```

```
# Model with an Embedding Layer
inputs = keras.Input(shape=(None,), dtype="int64")
embedded = layers.Embedding(input_dim=max_tokens, output_dim=128)(inputs)
X = layers.Bidirectional(layers.LSTM(32))(embedded)
X = layers.Dropout(0.2)(X) # Changed 'x' to 'X'
outputs = layers.Dense(1, activation="sigmoid")(X) # Changed 'x' to 'X'
embedding_model = keras.Model(inputs, outputs)
embedding_model.compile(optimizer="rmsprop",
                        loss="binary_crossentropy",
                        metrics=["accuracy"])
```

```
embedding_model. summary()
```

**Model: "functional"**

| Layer (type) | Output Shape | Param # |
|---|---|---|
| input_layer (InputLayer) | (None, None) | 0 |
| embedding (Embedding) | (None, None, 128) | 1,280,000 |
| bidirectional (Bidirectional) | (None, 64) | 41,216 |
| dropout (Dropout) | (None, 64) | 0 |
| dense (Dense) | (None, 1) | 65 |

**Total params:** 1,321,281 (5.04 MB)
**Trainable params:** 1,321,281 (5.04 MB)
**Non-trainable params:** 0 (0.00 B)

```
callbacks = [
    keras.callbacks.ModelCheckpoint("embedding_model.keras", save_best_only=Tru
]
```

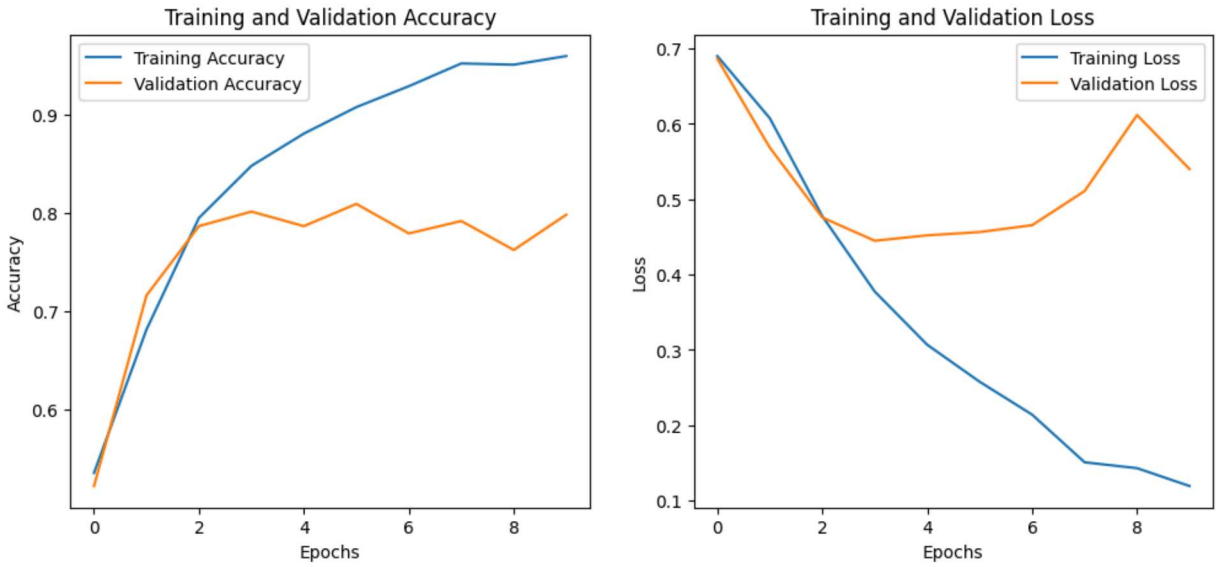
```
history_embedded = embedding_model.fit(
    int_train_ds,
```

```
    validation_data=int_val_ds,
    epochs=10,
    callbacks=callbacks)
```

```
Epoch 1/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 9s 44ms/step - accuracy: 0.5359 - loss: 0.6900 - va
Epoch 2/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 4s 35ms/step - accuracy: 0.6816 - loss: 0.6076 - va
Epoch 3/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 33ms/step - accuracy: 0.7950 - loss: 0.4776 - va
Epoch 4/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 33ms/step - accuracy: 0.8478 - loss: 0.3775 - va
Epoch 5/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 5s 52ms/step - accuracy: 0.8806 - loss: 0.3069 - va
Epoch 6/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 33ms/step - accuracy: 0.9075 - loss: 0.2578 - va
Epoch 7/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 32ms/step - accuracy: 0.9287 - loss: 0.2141 - va
Epoch 8/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 4s 41ms/step - accuracy: 0.9519 - loss: 0.1509 - va
Epoch 9/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 32ms/step - accuracy: 0.9506 - loss: 0.1431 - va
Epoch 10/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 35ms/step - accuracy: 0.9594 - loss: 0.1194 - va
```

```
import matplotlib.pyplot as plt
# Extract the history of metrics
history = history_embedded.history

# Plot training and validation accuracy
plt.figure(figsize=(12, 5))

# Subplot for accuracy
plt.subplot(1, 2, 1)
plt.plot(history['accuracy'], label='Training Accuracy')
plt.plot(history['val_accuracy'], label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

# Subplot for loss
plt.subplot(1, 2, 2)
plt.plot(history['loss'], label='Training Loss')
plt.plot(history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
```

```
<matplotlib.legend.Legend at 0x7885864a9c40>
```




```
# Show the graph
plt.tight_layout()
plt.show()
```

```
<Figure size 640x480 with 0 Axes>
```

```
# Model with Pretrained Word Embeddings

# Download Glove embeddings
!wget http://nlp.stanford.edu/data/glove.6B.zip
!unzip -q glove.6B.zip
```

```
--2026-04-18 19:48:59--  http://nlp.stanford.edu/data/glove.6B.zip
Resolving nlp.stanford.edu (nlp.stanford.edu)... 171.64.67.140
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:80... connected
HTTP request sent, awaiting response... 302 Found
Location: https://nlp.stanford.edu/data/glove.6B.zip [following]
--2026-04-18 19:48:59--  https://nlp.stanford.edu/data/glove.6B.zip
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:443... connecte
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip [following]
--2026-04-18 19:48:59--  https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip
Resolving downloads.cs.stanford.edu (downloads.cs.stanford.edu)... 171.64.64.22
Connecting to downloads.cs.stanford.edu (downloads.cs.stanford.edu)|171.64.64.22
HTTP request sent, awaiting response... 200 OK
Length: 862182613 (822M) [application/zip]
Saving to: ‘glove.6B.zip’

glove.6B.zip        100%[===================>] 822.24M  5.01MB/s    in 2m 39s

2026-04-18 19:51:38 (5.19 MB/s) - ‘glove.6B.zip’ saved [862182613/862182613]
```

```
# Prepare GloVe embedding matrix
embedding_dim = 100
path_to_glove_file = "glove.6B.100d.txt"

embeddings_index = {}
with open(path_to_glove_file) as f:
    for line in f:
        word, coefs = line.split(maxsplit=1)
        coefs = np.fromstring(coefs, "f", sep=" ")
        embeddings_index[word] = coefs

vocabulary = text_vectorization.get_vocabulary()
word_index = dict(zip(vocabulary, range(len(vocabulary))))

embedding_matrix = np.zeros((max_tokens, embedding_dim))
for word, i in word_index.items():
    if i < max_tokens:
        embedding_vector = embeddings_index.get(word)
        if embedding_vector is not None:
            embedding_matrix[i] = embedding_vector
```

```
# Pretrained embedding model
embedding_layer = layers.Embedding(
    input_dim=max_tokens,
    output_dim=embedding_dim,
    embeddings_initializer=keras.initializers.Constant(embedding_matrix),
    trainable=False,
    mask_zero=True,
)
```

```
inputs = keras.Input(shape=(None,), dtype="int64")
embedded = embedding_layer(inputs)
X = layers.Bidirectional(layers.LSTM(32))(embedded)
X = layers.Dropout(0.2)(X)
outputs = layers.Dense(1, activation="sigmoid")(X)
pretrained_model = keras.Model(inputs, outputs)
```

```
inputs = keras.Input(shape=(None,), dtype="int64")
embedded = embedding_layer(inputs)
X = layers.Bidirectional(layers.LSTM(32))(embedded)
X = layers.Dropout(0.2)(X)
outputs = layers.Dense(1, activation="sigmoid")(X)
pretrained_model = keras.Model(inputs, outputs)
pretrained_model.compile(optimizer="rmsprop",
                         loss="binary_crossentropy",
                         metrics=["accuracy"])
```

```
pretrained_model.summary()
```

**Model: "functional_2"**

| Layer (type) | Output Shape | Param # | Connected to |
|---|---|---|---|
| input_layer_2 (InputLayer) | (None, None) | 0 | - |
| embedding_1 (Embedding) | (None, None, 100) | 1,000,000 | input_layer_2[0]… |
| not_equal_1 (NotEqual) | (None, None) | 0 | input_layer_2[0]… |
| bidirectional_2 (Bidirectional) | (None, 64) | 34,048 | embedding_1[1][0…<br>not_equal_1[0][0] |
| dropout_2 (Dropout) | (None, 64) | 0 | bidirectional_2[… |
| dense_2 (Dense) | (None, 1) | 65 | dropout_2[0][0] |

**Total params:** 1,034,113 (3.94 MB)
**Trainable params:** 34,113 (133.25 KB)

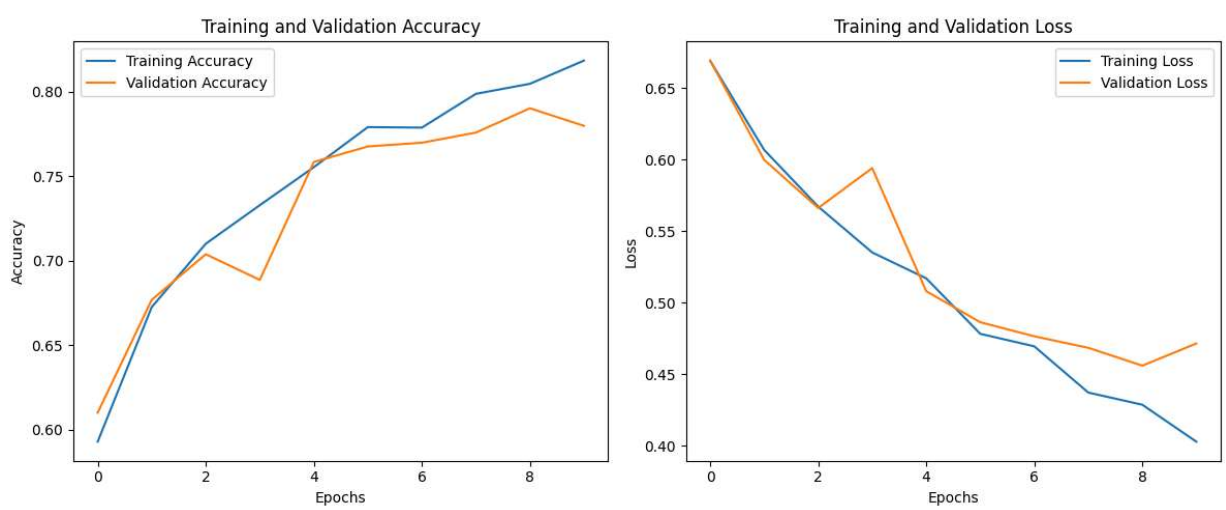
```
callbacks = [keras.callbacks.ModelCheckpoint ("pretrained_model.keras", save_be
]
```

```
history_pretrained = pretrained_model.fit(
    int_train_ds,
    validation_data=int_val_ds,
    epochs=10,
    callbacks=callbacks
)
```

```
Epoch 1/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 7s 53ms/step - accuracy: 0.5928 - loss: 0.6692 - va
Epoch 2/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 6s 65ms/step - accuracy: 0.6725 - loss: 0.6068 - va
Epoch 3/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 9s 50ms/step - accuracy: 0.7100 - loss: 0.5672 - va
Epoch 4/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 4s 41ms/step - accuracy: 0.7328 - loss: 0.5351 - va
Epoch 5/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 6s 59ms/step - accuracy: 0.7553 - loss: 0.5170 - va
Epoch 6/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 5s 51ms/step - accuracy: 0.7791 - loss: 0.4782 - va
Epoch 7/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 7s 65ms/step - accuracy: 0.7788 - loss: 0.4695 - va
Epoch 8/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 5s 47ms/step - accuracy: 0.7987 - loss: 0.4371 - va
```

```
Epoch 9/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 6s 57ms/step - accuracy: 0.8047 - loss: 0.4286 - va
Epoch 10/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 8s 33ms/step - accuracy: 0.8184 - loss: 0.4028 - va
```

```
import matplotlib.pyplot as plt

# Extract the history of metrics
history = history_pretrained.history

# Create a figure for the plots
plt.figure(figsize=(12, 5))

# Subplot for accuracy
plt.subplot(1, 2, 1)
plt.plot(history['accuracy'], label='Training Accuracy')
plt.plot(history['val_accuracy'], label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

# Subplot for loss
plt.subplot(1, 2, 2)
plt.plot(history['loss'], label='Training Loss')
plt.plot(history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

# Show the graph
plt.tight_layout()
plt.show()
```



```
import numpy as np
import matplotlib.pyplot as plt
import time

# Data
data_counts = [100, 200, 500, 1000]
custom_embed_scores = []
pretrained_embed_scores = []

# Initialize the plot
plt.figure(figsize=(12, 6))
plt.title('Model Accuracy vs. Training Sample Sizes')
plt.xlabel('Training Sample Size')
plt.ylabel('Accuracy')
plt.grid(True)

# Iterate over sample sizes
for i, size in enumerate(data_counts):
    print(f"\n### Training with {size} samples ###\n")
    # Update training dataset with the specific size
    small_train_ds = train_ds.map(lambda x, y: (text_vectorization(x), y)).take

    # Train custom embedding model
    print(f"Training Custom Embedding Model with {size} samples:")
    embedding_model.fit(
        small_train_ds,
        validation_data=int_val_ds,
        epochs=10,
        verbose=1
    )
    embedding_acc = embedding_model.evaluate(int_test_ds, verbose=1)[1]
    custom_embed_scores.append(embedding_acc)
    print(f"Custom Embedding Model Accuracy: {embedding_acc:.4f}\n")

    # Train pretrained embedding model
    print(f"Training Pretrained Embedding Model with {size} samples:")
    pretrained_model.fit(
        small_train_ds,
        validation_data=int_val_ds,
        epochs=10,
        verbose=1
    )

    pretrained_acc = pretrained_model.evaluate(int_test_ds, verbose=1)[1]
    pretrained_embed_scores.append(pretrained_acc)
    print(f"Pretrained Embedding Model Accuracy: {pretrained_acc:.4f}\n")

    # Update the plot dynamically after each iteration
    if i == len(data_counts) - 1:
        plt.plot(data_counts, custom_embed_scores, marker='o', label='Custom Em
        plt.plot(data_counts, pretrained_embed_scores, marker='o', label='Pretr
```

```
plt.legend()
plt.show()
```

```
### Training with 100 samples ###

Training Custom Embedding Model with 100 samples:
Epoch 1/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 31ms/step - accuracy: 0.9725 - loss: 0.0897 -
Epoch 2/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 5s 48ms/step - accuracy: 0.9819 - loss: 0.0578 -
Epoch 3/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 32ms/step - accuracy: 0.9787 - loss: 0.0663 -
Epoch 4/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 31ms/step - accuracy: 0.9781 - loss: 0.0643 -
Epoch 5/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 31ms/step - accuracy: 0.9909 - loss: 0.0299 -
Epoch 6/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 5s 48ms/step - accuracy: 0.9922 - loss: 0.0364 -
Epoch 7/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 31ms/step - accuracy: 0.9872 - loss: 0.0394 -
Epoch 8/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 29ms/step - accuracy: 0.9922 - loss: 0.0268 -
Epoch 9/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 28ms/step - accuracy: 0.9900 - loss: 0.0419 -
Epoch 10/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 4s 44ms/step - accuracy: 0.9972 - loss: 0.0113 -
782/782 ━━━━━━━━━━━━━━━━━━━━ 8s 10ms/step - accuracy: 0.7539 - loss: 0.8935
Custom Embedding Model Accuracy: 0.7539

Training Pretrained Embedding Model with 100 samples:
Epoch 1/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 34ms/step - accuracy: 0.8250 - loss: 0.3900 -
Epoch 2/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 4s 42ms/step - accuracy: 0.8322 - loss: 0.3804 -
Epoch 3/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 30ms/step - accuracy: 0.8556 - loss: 0.3485 -
Epoch 4/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 29ms/step - accuracy: 0.8578 - loss: 0.3304 -
Epoch 5/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 29ms/step - accuracy: 0.8725 - loss: 0.3100 -
Epoch 6/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 4s 45ms/step - accuracy: 0.8788 - loss: 0.2931 -
Epoch 7/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 29ms/step - accuracy: 0.8834 - loss: 0.2860 -
Epoch 8/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 29ms/step - accuracy: 0.9034 - loss: 0.2469 -
Epoch 9/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 3s 29ms/step - accuracy: 0.8978 - loss: 0.2435 -
Epoch 10/10
100/100 ━━━━━━━━━━━━━━━━━━━━ 4s 40ms/step - accuracy: 0.9156 - loss: 0.2170 -
782/782 ━━━━━━━━━━━━━━━━━━━━ 7s 9ms/step - accuracy: 0.7443 - loss: 0.6329
Pretrained Embedding Model Accuracy: 0.7443


### Training with 200 samples ###

Training Custom Embedding Model with 200 samples:
Epoch 1/10
```

```
200/200 ━━━━━━━━━━━━━━━━━━━━ 6s 30ms/step - accuracy: 0.8703 - loss: 0.3187 -
Epoch 2/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 4s 22ms/step - accuracy: 0.9156 - loss: 0.2155 -
Epoch 3/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 4s 22ms/step - accuracy: 0.9389 - loss: 0.1661 -
Epoch 4/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 6s 31ms/step - accuracy: 0.9545 - loss: 0.1359 -
Epoch 5/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 4s 22ms/step - accuracy: 0.9694 - loss: 0.0957 -
Epoch 6/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - accuracy: 0.9783 - loss: 0.0663 -
Epoch 7/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 7s 36ms/step - accuracy: 0.9844 - loss: 0.0522 -
Epoch 8/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 5s 24ms/step - accuracy: 0.9880 - loss: 0.0386 -
Epoch 9/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 6s 30ms/step - accuracy: 0.9917 - loss: 0.0251 -
Epoch 10/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 5s 26ms/step - accuracy: 0.9937 - loss: 0.0243 -
782/782 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.7857 - loss: 0.8799
Custom Embedding Model Accuracy: 0.7857

Training Pretrained Embedding Model with 200 samples:
Epoch 1/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 5s 25ms/step - accuracy: 0.8450 - loss: 0.3599 -
Epoch 2/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 6s 30ms/step - accuracy: 0.8434 - loss: 0.3555 -
Epoch 3/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 6s 29ms/step - accuracy: 0.8630 - loss: 0.3249 -
Epoch 4/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 5s 24ms/step - accuracy: 0.8741 - loss: 0.3030 -
Epoch 5/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 7s 33ms/step - accuracy: 0.8809 - loss: 0.2846 -
Epoch 6/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 5s 24ms/step - accuracy: 0.8916 - loss: 0.2713 -
Epoch 7/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 5s 25ms/step - accuracy: 0.9028 - loss: 0.2431 -
Epoch 8/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 6s 31ms/step - accuracy: 0.9077 - loss: 0.2321 -
Epoch 9/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - accuracy: 0.9161 - loss: 0.2113 -
Epoch 10/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 5s 27ms/step - accuracy: 0.9252 - loss: 0.1939 -
782/782 ━━━━━━━━━━━━━━━━━━━━ 8s 10ms/step - accuracy: 0.7827 - loss: 0.4927
Pretrained Embedding Model Accuracy: 0.7827


### Training with 500 samples ###

Training Custom Embedding Model with 500 samples:
Epoch 1/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 11s 22ms/step - accuracy: 0.8789 - loss: 0.2928 -
Epoch 2/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 11s 22ms/step - accuracy: 0.9099 - loss: 0.2246 -
Epoch 3/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 10s 20ms/step - accuracy: 0.9377 - loss: 0.1632 -
Epoch 4/10
```